ISE 537 Final Project Guide

Reinforcement Learning for Optimal Execution: A Multi-Agent Approach

---

The Central Argument

This project asks a single escalating question: *can a reinforcement learning agent discover good execution strategies from scratch, using only simulated experience calibrated to a real stock — and what happens when two such agents compete in the same market?*

Each of the four approaches answers a version of that question. Each answer reveals a limitation that motivates the next approach. The report is not four parallel experiments. It is one argument in four steps.

Abstract (write this last, but plan it now)

The abstract leads with the payoff. It should cover five things in roughly 150 words:

1. **The problem** — optimal execution under market impact
2. **The approach** — RL trained on a TSLA-calibrated simulation, escalating from tabular Q-learning to multi-agent DQN
3. **The Almgren-Chriss connection** — the single-agent policy rediscovers the closed-form solution from experience alone
4. **The headline result** — blind vs observable two-agent Sharpe comparison, quantifying the value of order flow transparency
5. **The punchline** — multi-agent competition is where RL earns its place, because no closed-form solution exists for that setting

Write the abstract after everything else is done. It is the one place in the report where you lead with the conclusion rather than the setup.

The Dataset

Asset: TSLA
Period: 2019–2023
Source: Yahoo Finance daily close prices
Why this period: It spans three distinct market regimes — pre-COVID calm, COVID stress, and

post-COVID recovery — giving natural variation in volatility that enriches the calibration and motivates the robustness discussion.

Two statistics to extract:

Daily volatility (σ) — the standard deviation of log returns. This becomes the noise term in the price process. Compute it from the full 2019–2023 period for the primary calibration, then note how it differs across sub-periods.

Market impact coefficient (λ) — derived from average daily volume using the Almgren-Chriss approximation $\lambda \approx \sigma / \text{ADV}$. TSLA trades roughly 100 million shares per day. A position of 1 million shares represents 1% of ADV — small enough to be realistic, large enough that impact is non-trivial.

What to say in the report: Every parameter in the simulation came from real data. That is the claim. Make it explicitly — do not let it read like the calibration was incidental.

The Simulation Environment

The base environment from the Jupyter notebook uses arbitrary parameters. This project replaces them with calibrated values. The price process is:

- `p_next = p + noise - λ · a²` where noise ~ $N(0, \sigma)$
- `p_trade = p - λ · a²`
- `reward = p_trade · a – ϕ · risk_penalty`

The risk penalty φ is essential. Without it the agent trivially learns to sell everything immediately, maximizing cash now and ignoring future periods. With it, the agent is forced to balance speed against impact — which is the actual execution problem. The penalty term should be proportional to the variance of remaining execution, following the Almgren-Chriss formulation.

What to say in the report: Explain why the risk penalty is necessary. Most students omit this. It is the mechanism that makes the learned policy interesting.

Approach 1: Tabular Q-Learning, Single Agent

What it is

The baseline. Implement the Q-learning algorithm from the Jupyter notebook with calibrated parameters. This satisfies the core rubric requirement and establishes the proof of concept.

State: (inventory remaining, time remaining, current price) — discretized into bins
Action: fraction of remaining inventory to sell this period — discretized into levels
Reward: cash received minus risk penalty
Constraint: all inventory must be sold by the deadline

Training

Run for several thousand episodes with ε-greedy exploration. Start at ε = 1.0 (pure exploration) and decay to ε = 0.01 (mostly exploitation). The decay schedule matters — decay too fast and the agent gets stuck in a suboptimal policy; too slow and training is inefficient.

What to show

Convergence plot: track Σ Q(s,a) across all state-action pairs over episodes. This sum starts noisy and large, then stabilizes as the Q-table converges. The flattening is your convergence evidence — required by the rubric.

Sharpe Ratio comparison against both benchmarks:

- *TWAP:* sell total_inventory / T each period. Simple, widely used in practice.
- *Execute at t=0:* sell everything immediately. Maximum impact, zero timing risk.

Sharpe here is computed across episodes: mean(total cash received) / std(total cash received). High Sharpe means the agent consistently gets good prices, not just occasionally lucky.

The closing sentence

End this section by exposing the limitation: the discrete state representation forces coarse binning of price and inventory. The agent cannot be sensitive to continuous price movements. This motivates the move to function approximation.

The Almgren-Chriss connection

This is the most important thing to say in this section. The agent — having never read the paper — will discover a policy that resembles the Almgren-Chriss solution: spread execution across time, sell slightly more when price is high, slightly less when it has dipped. Say this explicitly. Then explain what assumptions make it possible: quadratic impact, no private information, risk-penalized objective. This sets up the later argument that multi-agent competition breaks those assumptions.

Approach 2: DQN, Single Agent

What it is

Replace the Q-table with a neural network. This fixes the limitation exposed at the end of

Approach 1 — the state space becomes continuous, and the agent generalizes across states it has never seen.

Network architecture:

- Input: raw state vector (inventory, time, price) — 3 features
- Hidden: two layers, 64 units each, ReLU activation
- Output: Q-value for each discrete action

Replay buffer: store experience tuples (s, a, r, s') and sample random batches for training. This breaks temporal correlation in the training signal — a standard technique in production RL systems. The buffer should hold several thousand transitions before training begins.

Target network: maintain a second copy of the network with weights updated slowly. This stabilizes training by preventing the Q-value targets from shifting too rapidly.

What to show

Training loss curve alongside the convergence plot from Approach 1. The loss should decrease and stabilize — this is your DQN convergence evidence.

Sharpe comparison: DQN vs tabular Q-learning vs both benchmarks. If DQN wins, explain why — continuous state representation allows sensitivity to price levels that the table cannot capture. If they perform similarly, explain that too — the problem may be simple enough that coarse discretization suffices, which is itself an interesting finding.

Policy visualization: plot the learned action as a function of time remaining and inventory remaining. This should show a structured policy — more aggressive selling when inventory is high and time is short, more patient when time is ample. If it does, that is direct evidence the agent learned something financially sensible.

The closing sentence

End this section by exposing the remaining limitation: the single-agent assumption. The environment is stationary — it does not have opinions, it does not react strategically. In real markets, you are not the only participant liquidating. Other institutions are selling simultaneously, and their impact compounds with yours.

Approach 3: DQN, Two Agents, Blind

What it is

The core extension and primary novel contribution. Add a second agent with its own inventory, its own DQN, and its own replay buffer. Both agents are in the market simultaneously. At each

timestep, both choose an action, and the environment applies their *combined* impact to the price.

Combined impact: `p_next = p + noise - λ · (a_A + a_B)²`

Each agent receives its own reward — the cash it personally received. They are not cooperating. They are competing.

The blind design choice

Each agent observes only its own state: inventory remaining, time remaining, current price. It cannot see the other agent's inventory or last action. The other agent's behavior shows up only as unexplained price movements — prices move more than the agent's own impact explains, but it cannot tell why.

This is the realistic setting. In real markets, an execution desk cannot observe another institution's remaining order flow. You see the price. You do not see who is selling.

Agent asymmetry

Agent A holds twice the inventory of Agent B. This asymmetry is more realistic than a symmetric setup — institutional participants have different position sizes and different urgency. It also gives you something analytically interesting: does the larger agent learn to spread execution more carefully to avoid self-impact? Does the smaller agent learn to time around the larger agent's predictable pressure?

The inventory sizes should be randomized each episode within a range rather than fixed. This forces agents to develop robust policies — they cannot overfit to one specific opponent configuration. An agent that works well against any opponent size is a more credible result than one tuned to a single fixed setup.

Why this is hard

From each agent's perspective, the environment looks non-stationary. Prices move in ways the agent cannot fully account for, because the other agent is also learning and changing its behavior. Both agents are moving targets for each other. Convergence is harder to achieve and harder to prove than in the single-agent case.

Budget extra debugging time here. Early in training one agent may dominate — learning to sell aggressively while the other collapses to a passive strategy. This is a known failure mode in competitive multi-agent RL. If it happens, examine your reward scaling and exploration schedules for both agents independently.

What to show

Convergence: Q-value sums for both agents separately. Both should stabilize, though possibly at different rates given the inventory asymmetry.

Sharpe comparison: both agents vs TWAP vs execute-at-t=0 vs single-agent DQN. The multi-agent Sharpe for each agent will likely be lower than single-agent — competition degrades outcomes for both participants. If it does, say so. That is the correct financial finding: simultaneous liquidation is worse for everyone than sequential liquidation, which is why execution desks try to avoid crowded trades.

Policy comparison: compare Agent A's learned policy to Agent B's. Do they behave differently given their size difference? They should — the larger agent should be more conservative, the smaller agent more opportunistic.

The closing sentence

End this section by noting that agents trained blind must infer market conditions from price alone. This raises a natural question: how much does access to the opponent's information change behavior? That question motivates Approach 4.

Approach 4: DQN, Two Agents, Observable

What it is

Augment each agent's state with information about the other agent: their remaining inventory and their last action. Everything else stays identical to Approach 3. This is your comparison experiment.

State (observable): (own inventory, time, price, opponent inventory, opponent last action)

Agents can now behave strategically. Agent A can see that Agent B has large remaining inventory and anticipate aggressive selling. Agent B can see that Agent A is selling slowly and decide to front-run or hold back. The interaction becomes explicitly game-theoretic.

What to show

Sharpe comparison: observable vs blind for both agents. This is your headline result. It quantifies the value of order flow transparency in your simulated environment.

If observable agents achieve higher Sharpe, that means pre-trade transparency improves outcomes — knowing who is in the market and how much they have to trade allows better coordination, even among competitors. If the difference is small, that means price alone contains most of the relevant information, and formal transparency adds little. Either result is financially interesting and honestly reportable.

Policy differences: do observable agents behave differently from blind agents? Look for strategic behavior — one agent backing off when the other is aggressive, timing execution to avoid compounding impact. If you see it, describe it precisely. If you do not, that is also worth

discussing — convergence to similar policies despite different information sets would suggest the competitive dynamics dominate the strategic ones.

Results Table

Present this table as the centerpiece of your results section. Every row corresponds to one experiment. Every column is directly comparable.

Approach	Agents	Information	Sharpe A	Sharpe B	Notes
Execute at t=0	1	—	—	—	Lower bound
TWAP	1	—	—	—	Industry baseline
Tabular Q-learning	1	Own state	—	—	Rubric requirement
DQN	1	Own state	—	—	Function approximation
DQN blind	2	Own state only	—	—	Realistic competition
DQN observable	2	Own state + opponent	—	—	Transparent competition

Fill in the Sharpe values from your experiments. The progression of Sharpe ratios down the table tells the story of the project.

Financial Interpretation

Write more here than anywhere else. This is the section that separates good reports from great ones. The computational results are evidence for a financial argument — not the point themselves.

The Almgren-Chriss connection

The Almgren-Chriss model (2001) derives the optimal execution schedule analytically under quadratic market impact and a mean-variance objective. The solution is to spread execution across time, trading off impact cost against timing risk. The optimal schedule is deterministic and computable in closed form.

Your single-agent RL agent, having never seen this derivation, rediscovers something similar from experience. This is not a coincidence — both the analytic solution and the learned policy are responding to the same underlying tradeoff. The agent's policy is evidence that the simulation environment is well-specified: it is hard enough that naive strategies fail, structured enough that a good strategy is learnable.

Explicitly connect the learned policy to the Almgren-Chriss solution. Where do they agree? Where do they diverge? Divergence is interesting — it may reflect the risk penalty specification, the discretization of actions, or genuine differences between the model assumptions and the simulation dynamics.

Where RL earns its place

The Almgren-Chriss model has a clean closed-form solution precisely because it makes strong simplifying assumptions. Once you introduce a second agent, those assumptions break. The optimal policy for Agent A now depends on what Agent B does, and what Agent B does is itself a function of what Agent A does. There is no clean analytical solution to this fixed-point problem in general.

This is where RL earns its place. It does not need to solve the problem analytically. It discovers good policies through experience, even when the environment is too complex for a closed form. The multi-agent case is a concrete example of a setting where RL is not just a different way to get the same answer — it is the only tractable approach.

The transparency finding

The comparison between blind and observable agents maps to a real debate in market microstructure: how much does pre-trade transparency — formally knowing who is in the market and how much they have to trade — affect execution quality?

Your experiment gives a quantitative answer in a controlled simulated environment. Discuss the direction and magnitude of the Sharpe difference. Then discuss the limitations: your price process is a simplification, real order flow information is noisy and partial, and the strategic behavior of real participants is far more complex than two DQN agents. But the framework is right, and the direction of the finding is informative.

Limitations

Be honest about these. It strengthens the report rather than weakening it.

- The price process is a simplification. Real markets have bid-ask spreads, intraday volume patterns, and order book depth that changes continuously.
- The impact function is quadratic and symmetric. Real impact is more complex and has a transient component that partially recovers after a trade.
- Two agents is a toy version of a real market with hundreds of participants.

- Calibrating λ from daily close data is approximate. Tick-level data would give a better estimate.

Then note what this project does right despite those limitations: parameters are grounded in real data, evaluation uses a financially meaningful metric, and the experimental design is controlled enough to isolate the effect of each design choice.

Report Structure and Page Allocation

The rubric requires 3–10 pages. Target 8–9 pages to show thoroughness without padding.

Abstract — 150 words. Lead with the payoff. Write this last.

Introduction — 1 page. The execution problem, TSLA as the asset, the period and why it was chosen, calibration methodology. No mention of DQN or multi-agent yet. Just the problem and the data.

Model Introduction — 1.5 pages. Q-learning: Bellman equation, ε-greedy, Q-table update. DQN: network approximator, replay buffer, target network. Multi-agent setup: shared environment, combined impact, independent rewards, blind vs observable state definitions. Keep this tight — it is background, not the contribution.

Parameter Selection — 1 page. One paragraph per meaningful choice: ε decay schedule, replay buffer size, network architecture, number of episodes, inventory asymmetry between agents, risk penalty coefficient. Explain the reasoning for each choice, not just the value.

Results — 2 pages. The six-row comparison table. Convergence plots for single-agent and multi-agent. Policy visualization for at least one agent. The blind vs observable comparison. Let the figures do the work — write tight captions that interpret rather than just describe.

Financial Interpretation — 2 pages. Almgren-Chriss connection. Where RL earns its place. The transparency finding. Limitations. This is your thesis. Write it like one.

Conclusion — 0.5 pages. One paragraph summarizing the methodology and the progression of results. One sentence on limitations. One sentence on what you would do next with more time — multi-agent with more than two agents, calibration from tick-level data, or a more realistic impact model.

The Closing Sentence for Each Section

These sentences do the structural work of the entire report. Draft them before writing the sections themselves.

End of tabular Q-learning: *"While the learned policy recovers the Almgren-Chriss intuition from experience alone, the discrete state representation limits the agent's sensitivity to continuous price movements, motivating a function approximation approach."*

End of single-agent DQN: *"The DQN agent generalizes effectively across continuous state spaces, but retains the single-agent assumption that the trader operates in isolation — an assumption that real execution desks cannot make."*

End of blind two-agent: *"Training under full uncertainty produces a robust policy, but raises a natural question: how much does access to the opponent's position change execution behavior and outcomes?"*

End of observable two-agent: *"The difference in Sharpe between blind and observable agents quantifies, within this simulated environment, the execution value of pre-trade order flow transparency."*

Time Estimate

Task	Hours
Data pull and calibration	2–3
Single-agent environment	2–3
Tabular Q-learning	3–4
Benchmarks	1–2
DQN implementation	4–6
Multi-agent environment	3–4
Blind training and debugging	4–6
Observable experiment	2–3
Report writing	6–8
Total	**27–39 hours**

Give yourself two weeks. Week 1 is code — Stage 1 first, then DQN, then multi-agent. Week 2 is experiments, results, and writing. The multi-agent convergence debugging is the biggest wildcard — budget a full afternoon for it and do not be surprised if it takes longer.

Key References to Read Before Writing

Almgren and Chriss (2001) — *Optimal Execution of Portfolio Transactions*. The paper your single-agent agent implicitly rediscovers. Read at least the problem formulation and the optimal solution. You will cite this in the financial interpretation section.

Spooner et al. (2018) — *Market Making via Reinforcement Learning*. Good background on RL applied to execution problems with a similar simulation approach.

Recent Advances in Reinforcement Learning in Finance (SSRN 3971071) — the review paper listed in the project rubric. Read this. It contextualizes your project within the broader literature and gives you language to use in the introduction.

This document is a planning guide. The experiments may produce unexpected results — if they do, report them honestly and interpret them carefully. Unexpected findings, explained well, are more valuable than expected findings reported without insight.